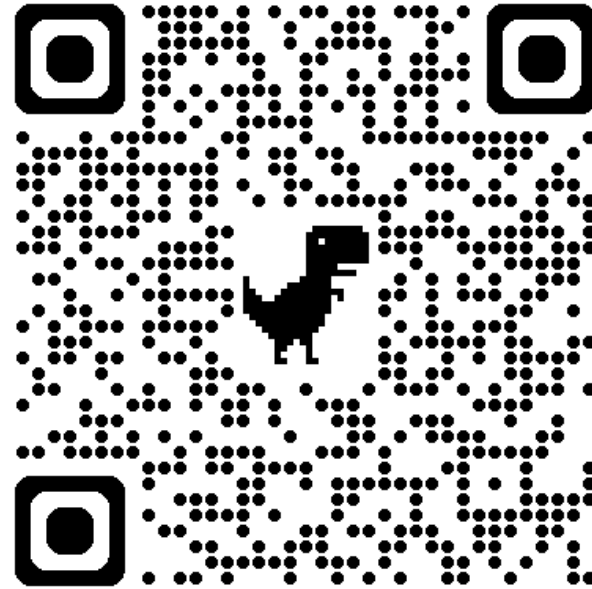


10 things you didn't know about Jetpack Compose modifiers

Thomas Künneth





<https://github.com/tkuenneth/ModifierDemo>

A modifier that keeps the device screen on as long as it is part of the composition on supported platforms.

This is useful for scenarios where the user might not be interacting with the screen frequently but the content needs to remain visible, such as during video playback.

```
fun Modifier.keepScreenOn(): Modifier = this then KeepScreenOnElement
```

```
1 @Composable
2 fun KeepScreenOnDemo() {
3     var keepScreenOnActive by remember { mutableStateOf(false) }
4     SwitchWithText(
5         active = keepScreenOnActive,
6         text = stringResource(R.string.keep_screen_on),
7         modifier = if (keepScreenOnActive) Modifier.keepScreenOn() else Modifier
8     ) {
9         keepScreenOnActive = it
10    }
11 }
```

```

32 private data object KeepScreenOnElement : ModifierNodeElement<KeepScreenOnNode>() {
33     override fun create(): KeepScreenOnNode = KeepScreenOnNode()
34
35     override fun update(node: KeepScreenOnNode) {}
36
37     override fun InspectorInfo.inspectableProperties() {
38         name = "keepScreenOn"
39     }
40 }
41
42 private class KeepScreenOnNode : Modifier.Node() {
43     override fun onAttach() {
44         requireOwner().incrementKeepScreenOnCount()
45     }
46
47     override fun onDetach() {
48         requireOwner().decrementKeepScreenOnCount()
49     }
50 }

```

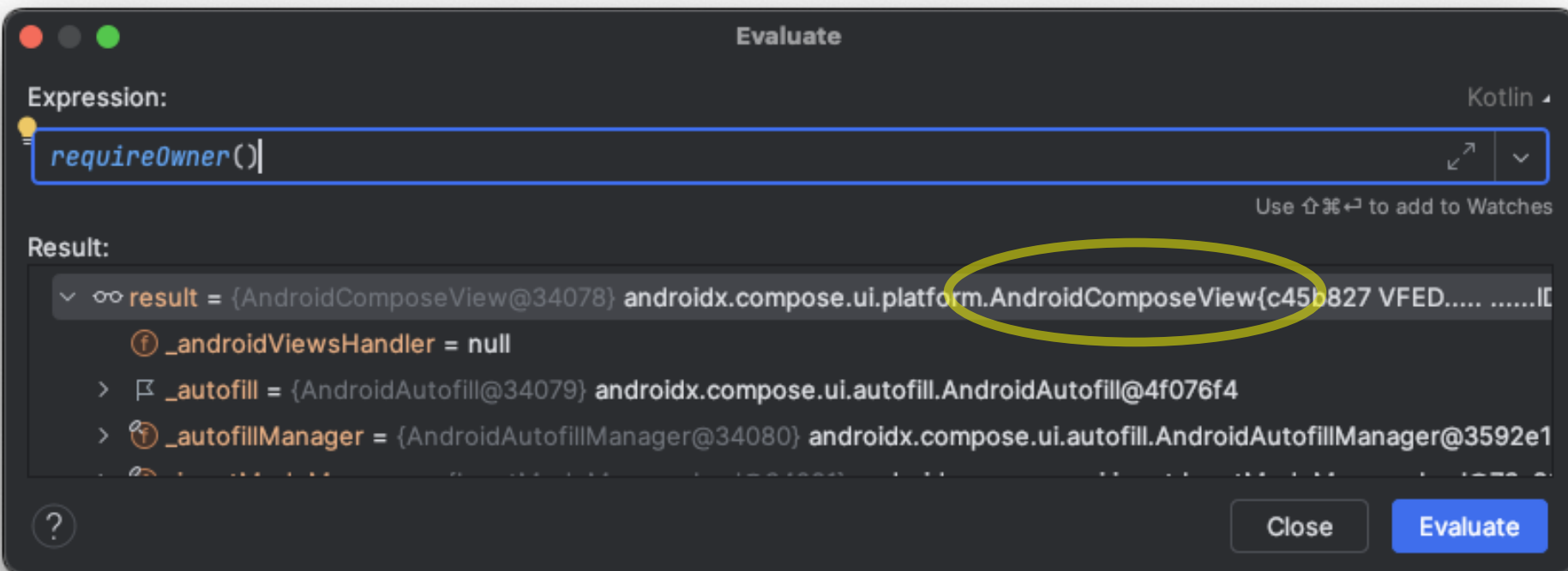
```
public abstract class Node : DelegatableNode
```

```
internal fun DelegatableNode.requireOwner(): Owner =  
    checkPreconditionNotNull( value = requireLayoutNode().owner) { "This node does not have an owner." }
```

```
internal fun DelegatableNode.requireLayoutNode(): LayoutNode =  
    checkPreconditionNotNull( value = node.coordinator) {  
        "Cannot obtain node coordinator. Is the Modifier.Node attached?"  
    }  
    .layoutNode
```

An element in the layout hierarchy, built with compose UI.

```
@OptIn( ...markerClass = InternalComposeUiApi::class)  
internal class LayoutNode(
```

```
@Suppress( ...names = "ViewConstructor", "VisibleForTests", "NullAnnotationGroup")
@OptIn( ...markerClass = InternalComposeUiApi::class)
internal class AndroidComposeView(context: Context, coroutineContext: CoroutineContext) :
    ViewGroup(context),
    Owner,
    PlatformFocusOwner,
    ViewRootForTest,
    MatrixPositionCalculator,
    DefaultLifecycleObserver,
    OutOfFrameExecutor {
```

Owner implements the connection to the underlying view system. On Android, this connects to Android `views` and all layout, draw, input, and accessibility is hooked through them.

```
internal interface Owner : PositionCalculator {
```

Increments count of modifiers requesting to stop the screen from going to sleep

```
fun incrementKeepScreenOnCount() {}
```

Decrements count of modifiers requesting to stop the screen from going to sleep

```
fun decrementKeepScreenOnCount() {}
```

```
private var keepScreenOnCount = 0

override fun incrementKeepScreenOnCount() {
    keepScreenOnCount++
    view.keepScreenOn = keepScreenOnCount > 0
}

override fun decrementKeepScreenOnCount() {
    keepScreenOnCount--
    view.keepScreenOn = keepScreenOnCount > 0
}
```

```
public void setKeepScreenOn(boolean keepScreenOn) {
    setFlags(keepScreenOn ? KEEP_SCREEN_ON : 0, KEEP_SCREEN_ON);
}
```


Things to remember

- `keepScreenOn()` keeps the screen on
- Modifier chains can be built using `ModifierNodeElements`, which in turn use `Modifier.Node` implementations
- Besides being a bridge between Compose and Views, `AndroidComposeView` makes available several View-based features, e.g. keeping the screen on

- `ModifierNodeElement` acts as a stateless factory; its `create()` / `update()` functions create and manage `Modifier.Node` instances
- `Modifier.Node` instances define the behavior; they hold state and are efficiently reused across recompositions
- `Modifier.Node.onAttach()` / `onDetach()` are lifecycle callbacks for when the node joins or leaves the UI tree
- Logic inside `Modifier.Node` can run independently, bypassing the overhead of recomposition



```
1  @Composable
2  fun UriHandlerDemo() {
3      val uriHandler = LocalUriHandler.current
4      val url = stringResource(R.string.url)
5      Text(
6          text = url,
7          modifier = Modifier
8              .clickable {
9                  uriHandler.openUri(url)
10             }
11     )
12 }
```

```
24 class AndroidUriHandler(private val context: Context) : UriHandler {
25
    Open given URL in browser
    Throws: IllegalArgumentException - when given uri is invalid and/or can't be handled by
           the system
32  override fun openUri(uri: String) {
33     try {
34         context.startActivity(Intent(action = Intent.ACTION_VIEW, Uri.parse(uriString = uri)))
35     } catch (e: ActivityNotFoundException) {
36         throw IllegalArgumentException(p0 = "Can't open $uri.", p1 = e)
37     }
38 }
39 }
```



```
1 @Composable
2 fun UriHandlerDemo() {
3     val uriHandler = LocalUriHandler.current
4     val url = stringResource(R.string.url)
5     Text(
6         text = url,
7         modifier = Modifier
8             .clickable {
9                 uriHandler.openUri(url)
10            }
11 }
12 }
```

try-catch or other
saftey nets



```
1 .clickable {
2     try {
3         uriHandler.openUri(url)
4     } catch (_: IllegalArgumentException) {
5         // Do something
6     }
7 }
```



Calls the specified function `block` and returns its encapsulated result if invocation was successful, catching any `Throwable` exception that was thrown from the `block` function execution and encapsulating it as a failure.

```
@InlineOnly
@SinceKotlin( version = "1.3")
public inline fun <R> runCatching(block: () -> R): Result<R> {
    return try {
        Result.success( value = block())
    } catch (e: Throwable) {
        Result.failure( exception = e)
    }
}
```

```
1  .clickable {
2      runCatching {
3          uriHandler.openUri(uri)
4      }.onFailure {
5          // do something
6      }
7  }
```



Things to remember

- Many hate Javas checked exceptions, but ...
- Always read APIs to learn about **exceptions**, and things happening **under the hood**
- Use Compose-provided mechanisms when they make sense, not *just because* they are there
- Separation of Concerns: ViewModels were invented to get business logic out of UI code; always consider doing stuff there before keeping them inside composables

```

fun Modifier.clickable(
    enabled: Boolean = true,
    onClickLabel: String? = null,
    role: Role? = null,
    interactionSource: MutableInteractionSource? = null,
    onClick: () -> Unit,
): Modifier {
    @OptIn(markerClass = ExperimentalFoundationApi::class)
    return if (ComposeFoundationFlags.isNonComposedClickableEnabled) {
        this.then(
            other = ClickableElement(
                interactionSource = interactionSource,
                indicationNodeFactory = null,
                useLocalIndication = true,
                enabled = enabled,
                onClickLabel = onClickLabel,
                role = role,
                onClick = onClick,
            )
        )
    } else {
        composed(
            inspectorInfo =
                debugInspectorInfo {
                    name = "clickable"
                    properties["enabled"] = enabled
                    properties["onClickLabel"] = onClickLabel
                    properties["role"] = role
                    properties["interactionSource"] = interactionSource
                    properties["onClick"] = onClick
                }
        )
    }
}

```

This is a collection of flags which are used to guard against regressions in some of the "riskier" refactors or new feature support that is added to this module. These flags are always "on" in the published artifact of this module, however these flags allow end consumers of this module to toggle them "off" in case this new path is causing a regression.

These flags are considered temporary, and there should be no expectation for these flags be around for an extended period of time. If you have a regression that one of these flags fixes, it is strongly encouraged for you to file a bug ASAP.

* In order to turn a feature off in a debug environment, it is recommended to set this to false in
* as close to the initial loading of the application as possible. Changing this value after compose
* library code has already been loaded can result in undefined behavior.



```
1 class MyApplication : Application() {  
2     override fun onCreate() {  
3         ComposeFoundationFlags.SomeFeatureEnabled = false  
4         super.onCreate()  
5     }  
6 }
```



```
1 -assumevalues class androidx.compose.runtime.ComposeFoundationFlags {  
2     public static boolean SomeFeatureEnabled return false  
3 }
```

* In order to turn this off in a release environment, it is recommended to additionally utilize R8
* rules which force a single value for the entire build artifact. This can result in the new code
* paths being completely removed from the artifact, which can often have nontrivial positive
* performance impact.

Things to remember

- `ComposeFoundationFlags` activates new, unstable APIs in `compose.foundation`
- Previews upcoming behaviors for core UI components
- Provides on/off switches for specific, in-development features
- Don't toggle them, unless you really have to



```
1 fun Modifier.openUrl(  
2     url: String,  
3     onFailure: (Throwable) -> Unit = {}  
4 ): Modifier =  
5     this then OpenUrlModifierNodeElement(  
6         url = url,  
7         onFailure = onFailure  
8     )
```



```
1 private data class OpenUrlModifierNodeElement(  
2     val url: String,  
3     val onFailure: (Throwable) -> Unit  
4 ) : ModifierNodeElement<OpenUrlModifierNode>() {  
5     override fun create() = OpenUrlModifierNode(  
6         initialUrl = url,  
7         initialOnFailure = onFailure  
8     )  
9  
10    override fun update(node: OpenUrlModifierNode) {  
11        node.url = url  
12        node.onFailure = onFailure  
13    }  
14  
15    override fun InspectorInfo.inspectableProperties() {  
16        name = "openUrl"  
17        properties["url"] = url  
18        properties["onFailure"] = onFailure  
19    }  
20 }
```

A `Modifier.Node` which is able to delegate work to other `Modifier.Node` instances.
This can be useful to compose multiple node implementations into one.

Implementing this interface allows your `Modifier.Node` subclass to read `CompositionLocals` via the `currentValueOf` function. The values of each `CompositionLocal` will be resolved based on the context of the layout node that the modifier is attached to, meaning that the modifier will see the same values of each `CompositionLocal` as its corresponding layout node.

Restart the gesture detection coroutine to force it to capture and use the latest state, such as an updated URL or `onFailure` callback

```
1 private class OpenUrlModifierNode(
2     initialUrl: String,
3     initialOnFailure: (Throwable) -> Unit
4 ) : DelegatingNode(), CompositionLocalConsumerModifierNode {
5
6     private val pointerInputNode = SuspendingPointerInputModifierNode {
7         detectTapGestures {
8             runCatching {
9                 uriHandler?.openUri(url)
10            }.onFailure { onFailure(it) }
11        }
12    }
13
14    var url: String = initialUrl
15        set(value) {
16            if (field != value) {
17                field = value
18                pointerInputNode.resetPointerInputHandler()
19            }
20        }
21
22    var onFailure: (Throwable) -> Unit = initialOnFailure
23        set(value) {
24            if (field != value) {
25                field = value
26                pointerInputNode.resetPointerInputHandler()
27            }
28        }
29
30    private val uriHandler: UriHandler?
31        get() = currentValueOf(LocalUriHandler)
32
33    init { delegate(pointerInputNode) }
34 }
```

Things to remember

- Compose complex modifiers by **delegating** behavior to other specialized nodes
- Reuse powerful base nodes like `LayoutModifierNode` or `DrawModifierNode` to **separate** concerns from your core logic
- **Combine** multiple behaviors by delegating to several nodes at once (e.g. `PointerInputModifierNode` for input and `SemanticsModifierNode` for accessibility)
- For certain nodes, control their lifecycle by calling specific methods (e.g., `resetPointerInputHandler()` for gesture detection)

10:34

ModifierDemo

☐ Magnifier

☐ Keep screen on

<https://github.com/tkuenneth/ModifierDemo>

Interact with this text to see context menus in action)

e quick brown

e quick brown

```
1 var magnifierActive by remember { mutableStateOf(false) }
2 Column(
3     modifier = Modifier
4         .draggableMagnifier(enabled = magnifierActive)
5         .fillMaxSize()
6         .padding(innerPadding)
7         .padding(horizontal = 16.dp),
8     verticalArrangement = Arrangement.spacedBy(16.dp)
9 ) {
10     MagnifierDemo(magnifierActive = magnifierActive) { magnifierActive = it }
11     ...
12 }
```

```
1 @Composable
2 fun MagnifierDemo(magnifierActive: Boolean, magnifierActiveChange: (Boolean) -> Unit) {
3     SwitchWithText(
4         active = magnifierActive,
5         text = stringResource(R.string.magnifier),
6         onActiveChange = magnifierActiveChange
7     )
8 }
```



```

1 fun Modifier.draggableMagnifier(enabled: Boolean): Modifier = composed {
2     if (enabled) {
3         var touchPosition by remember { mutableStateOf(Offset.Unspecified) }
4         pointerInput(Unit) {
5             detectDragGestures(
6                 onDragStart = { touchPosition = it },
7                 onDrag = { _, delta -> touchPosition += delta },
8                 onDragEnd = { touchPosition = Offset.Unspecified },
9                 onDragCancel = { touchPosition = Offset.Unspecified }
10            )
11        }.magnifier(
12            sourceCenter = { touchPosition },
13            magnifierCenter = { touchPosition },
14            zoom = 4F
15        )
16    } else {
17        Modifier
18    }
19 }

```

- sourceCenter: location on the original content to zoom in on
- magnifierCenter: center of the magnifier lens
- zoom: magnification level; values greater than 1f zoom in

```

internal fun Modifier.magnifier(
    sourceCenter: Density.() -> Offset,
    magnifierCenter: (Density.() -> Offset)? = null,
    onSizeChanged: ((DpSize) -> Unit)? = null,
    zoom: Float = Float.NaN,
    useTextDefault: Boolean = false,
    size: DpSize = DpSize.Unspecified,
    cornerRadius: Dp = Dp.Unspecified,
    elevation: Dp = Dp.Unspecified,
    clippingEnabled: Boolean = true,
    platformMagnifierFactory: PlatformMagnifierFactory? = null,
): Modifier {
    return if (isPlatformMagnifierSupported()) {
        then(
            other = MagnifierElement(
                sourceCenter = sourceCenter,
                magnifierCenter = magnifierCenter,
                onSizeChanged = onSizeChanged,
                useTextDefault = useTextDefault,
                zoom = zoom,
                size = size,
                cornerRadius = cornerRadius,
                elevation = elevation,
                clippingEnabled = clippingEnabled,
                platformMagnifierFactory =
                    platformMagnifierFactory
                    ?: PlatformMagnifierFactory
                        .getForCurrentPlatform(), // this doesn't do an alloc
            )
        )
    } else {
        // Magnifier is only supported in >=28. So avoid doing all the work to manage the magnifier
        // state if it's not needed.
        // TODO(b/202739980) Investigate supporting Magnifier on earlier versions.
        this
    }
}

```

```
@ChecksSdkIntAtLeast(api = 28)
```

```
internal fun isPlatformMagnifierSupported(sdkVersion: Int = Build.VERSION.SDK_INT) =
    sdkVersion >= 28
```

Things to remember

- `magnifier()` uses `Android Magnifier` widget under the hood (introduced in API level 28)
- No public Compose APIs for querying the availability of the magnifier
- Don't put SDK version checks in your `Modifier` chain

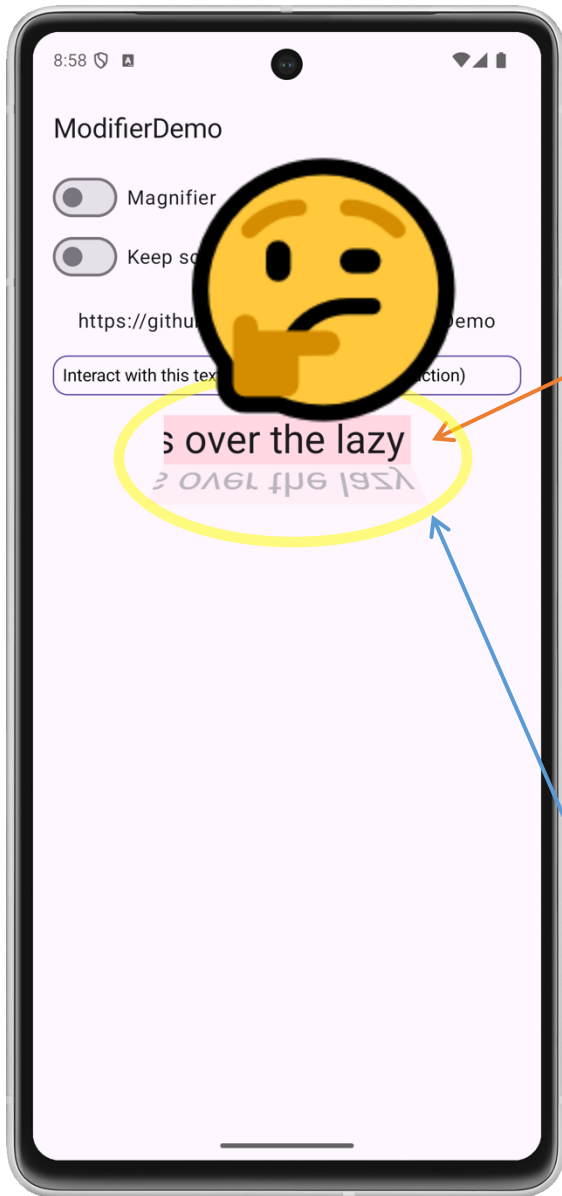


```
1 @Composable
2 fun ContextMenuDemo(text: String) {
3     val state = rememberTextFieldState(text)
4     val context = LocalContext.current
5     Box(
6         modifier = Modifier
7             .fillMaxWidth()
8             .border(
9                 width = 1.dp,
10                color = MaterialTheme
11                shape = MaterialTheme
12            )
13        .padding(all = 8.dp)
14    ) {
15        BasicTextField(
16            state = state,
17            modifier = Modifier
18                .appendTextContextMenuComponents {
19                    item(
20                        key = ContextMenuComponentsKey.Clear,
21                        label = context.getString(R.string.clear)
22                    ) {
23                        state.clearText()
24                        close()
25                    }
26                }
27            .filterTextContextMenuComponents { component ->
28                if (component.key == ContextMenuComponentsKey.Clear) state.text.isNotEmpty() else true
29            },
30        )
31    }
32 }
```

```
1 sealed class ContextMenuComponentsKey {
2     data object Clear : ContextMenuComponentsKey()
3 }
```

Things to remember

- `BasicTextField()` can show a context menu
- `appendTextContextMenuComponents()` provides additional items
- `filterTextContextMenuComponents()` decides which ones will be visible



```
1 @Composable
2 fun TextWithMarquee() {
3     Text(
4         text = stringResource(R.string.pangram),
5         modifier = Modifier
6             .width(200.dp)
7             .background(color = MaterialTheme.colorScheme.tertiaryContainer)
8             .basicMarquee(),
9         style = MaterialTheme.typography.headlineLarge
10    )
11 }
```



```
1 @Composable
2 fun GraphicsLayerDemo() {
3     WithReflection(
4         modifier = Modifier.graphicsLayer {
5             scaleX = 1f
6             scaleY = -1f
7             alpha = 0.3f
8             rotationX = 45F
9             transformOrigin = TransformOrigin(pivotFractionX = 0.5f, pivotFractionY = 1f)
10        }) { TextWithMarquee() }
11 }
```



```
1  @Composable
2  fun WithReflection(
3      modifier: Modifier, content: @Composable () -> Unit
4  ) {
5      Box(
6          contentAlignment = Alignment.Center
7      ) {
8          content()
9          Box(modifier = modifier) {
10             content()
11         }
12     }
13 }
```

```

1  @Composable
2  fun GraphicsLayerDemo2() {
3      val graphicsLayer = rememberGraphicsLayer()
4      Text(
5          text = stringResource(R.string.pangram),
6          modifier = Modifier
7              .drawWithContent {
8                  graphicsLayer.record {
9                      this@drawWithContent.drawContent()
10                 }
11                 drawContent()
12                 with(graphicsLayer) {
13                     scaleX = 1f
14                     scaleY = -1f
15                     alpha = 0.3f
16                     rotationX = 45F
17                     pivotOffset = Offset(size.width / 2F, size.height.toFloat())
18                     drawLayer(this)
19                 }
20             }
21         .width(200.dp)
22         .background(color = MaterialTheme.colorScheme.tertiaryContainer)
23         .basicMarquee(),
24         style = MaterialTheme.typography.headlineLarge
25     )
26 }

```


Things to remember

- `graphicsLayer()` makes content draw into a draw layer
- can be used to apply effects to content, such as scaling, rotation, opacity, shadow, and clipping
- Low-level API for advanced use cases:
 - `record()` saves the current state of the draw layer into a bitmap
 - `drawLayer()` draws the layer



```
1 fun Modifier.draggableMagnifier(enabled: Boolean): Modifier = composed {  
2     if (enabled) {  
3         var touchPosition by remember { mutableStateOf(Offset.Unspecified) }  
4         pointerInput(Unit) {  
5             detectDragGestures(  
6                 onDragStart = { touchPosition = it },  
7                 onDrag = { _, delta -> touchPosition += delta },  
8                 onDragEnd = { touchPosition = Offset.Unspecified },  
9                 onDragCancel = { touchPosition = Offset.Unspecified }  
10            )  
11        }.magnifier(  
12            sourceCenter = { touchPosition },  
13            magnifierCenter = { touchPosition },  
14            zoom = 4F  
15        )  
16    } else {  
17        Modifier  
18    }  
19 }
```



```
1 @Composable  
2 fun KeepScreenOnDemo() {  
3     var keepScreenOnActive by remember { mutableStateOf(false) }  
4     SwitchWithText(  
5         active = keepScreenOnActive,  
6         text = stringResource(R.string.keep_screen_on),  
7         modifier = if (keepScreenOnActive) Modifier.keepScreenOn() else Modifier  
8     ) {  
9         keepScreenOnActive = it  
10    }  
11 }
```

```
1 inline fun Modifier.conditional(  
2     condition: Boolean,  
3     block: Modifier.() -> Modifier  
4 ): Modifier {  
5     return if (condition) {  
6         then(block())  
7     } else {  
8         this  
9     }  
10 }
```

```
1 @Composable  
2 fun KeepScreenOnDemo() {  
3     var keepScreenOnActive by remember { mutableStateOf(false) }  
4     SwitchWithText(  
5         active = keepScreenOnActive,  
6         text = stringResource(R.string.keep_screen_on),  
7         modifier = Modifier.conditional(keepScreenOnActive) { keepScreenOn() }  
8     ) {  
9         keepScreenOnActive = it  
10    }  
11 }
```

Things to remember

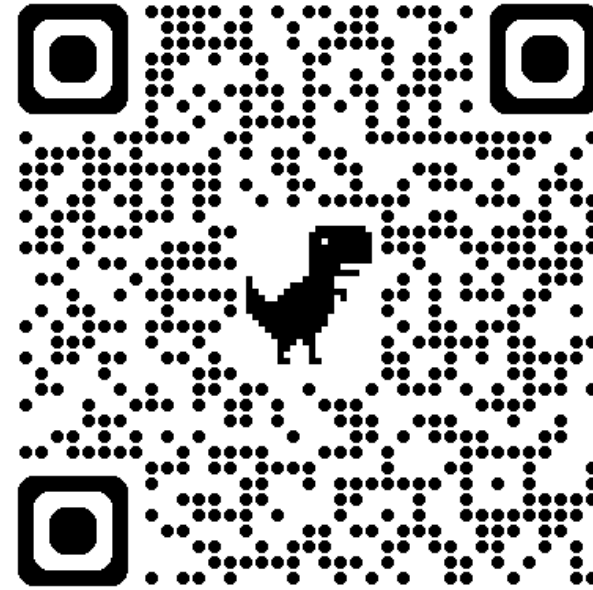
- No `Modifier.if()` by design: Compose expects you to use Kotlin's native `if/else`
- Custom conditionals can improve readability
- Consider `inline` extension functions for clean code without performance cost

Thank you!

 @tkuenneth

 @tkuenneth

 @tkuenneth.dev



<https://github.com/tkuenneth/ModifierDemo>